study · picasso and Determinate Systems · h0ffmann/picasso#96

Borrow the platform, keep the labs: picasso and Determinate Nix

picasso already stands on Determinate Systems' work: its workstation runs Determinate Nix, its README installs it, and every CI job starts with their installer Action. This study puts what Determinate Systems sells and gives away next to what picasso is, side by side, and asks one practical question: what can a one-person flake repository take from that platform, and add on top of it, between October 2026 and April 2027?

Matheus Hoffmann · Escola Politécnica, Universidade Federal do Rio de Janeiro · version 1.1, 12 October 2026

Scope. Every claim about Determinate Systems comes from its public product pages, documentation, blog or GitHub repositories, fetched on 12 October 2026 and numbered in §09. Performance figures are the vendor's and were not measured here. Every claim about picasso names a file at commit `140dc91`. Drafted with the assistance of Claude Code (Anthropic) and reviewed by the author.



Summary

The relation. Determinate Systems and picasso are not on the same layer. Determinate ships a Nix distribution (a fork of NixOS/nix under LGPL-2.1 [2]), an installer, a flake registry with semantic versioning (FlakeHub), a paid binary cache and a paid, SLA-backed package set [1, 3, 6, 9]. picasso is six self-contained flakes, called labs, that pin toolchains for one person's work (wave modelling, PDFs, linting, agents, CUDA, presenting) and that other repositories borrow [16]. picasso is a **customer** of the first layer, not a competitor to it.

What we found. picasso depends on Determinate in four places (§03). Two of them are stale: the publisher Action pins `nix-installer-action@v16` and `magic-nix-cache-action@v9`, a release from January 2025 that predates the cache's June 2025 revival [11, 12]. Five of the six lab locks are older than the 30 days that Determinate's free `flake-checker` flags [13].

Six months. Seven items, all free tiers or open-source tools, sized for one person's evenings (§05). The core is: fix the stale Action, add the free checks, publish the labs to FlakeHub as public flakes on each release tag, and keep picasso's own differentiators, which are domain toolchains, a bit-for-bit rebuild check and citable releases. Things picasso will not do are listed too: an SLA, FIPS, SOC 2, a private cache, a Nix fork.

01 two different layers

Determinate Systems describes Determinate Nix as its "validated and secure downstream distribution of NixOS/nix designed for enterprise use cases" [2]. The product page sells it on speed (lazy trees, parallel evaluation), on operations (managed garbage collection, MDM roll-out, OpenTelemetry traces) and on compliance (SOC 2 Type II; the footer adds FedRAMP High) [1]. Around the distribution sit FlakeHub, a registry for flakes, and FlakeHub Cache and Determinate Secure Packages, which are paid [6, 8, 9].

picasso's AGENTS.md describes itself as "self-contained Nix flakes ('labs') for the toolchains I work in, on an Ubuntu workstation (i9 + RTX 4090) running Determinate Nix — not NixOS" [16]. Its unit is the lab: one directory with its own `flake.nix`, lock, `justfile`, README and `lab.json`, that references nothing outside itself, so that ww-lab can check out `labs/pratico` as a sparse submodule and marola can pin `?dir=labs/lint`.

the honest frame

Determinate Systems is a company with a staff, a compliance programme and enterprise customers. picasso is one person's repository with CI on free GitHub-hosted runners, two downstream consumers (ww-lab, marola) and the profile README. Nothing in this study proposes that picasso match the vendor's scope. The useful question is narrower: which of the vendor's *free* parts make the labs better, and what do the labs give that a general platform does not.

02 what Determinate Systems offers

component	what it does, in the vendor's words or close to them	cost	src
Determinate Nix	Downstream Nix, "fully compatible with upstream Nix", plus parallel evaluation, lazy trees, a native Linux builder for macOS, WebAssembly in expressions, `nix ps`, OpenTelemetry tracing and stable flakes on by default. Custom settings go only in `/etc/nix/nix.custom.conf`.	free (LGPL-2.1)	[1, 2, 3]

component	what it does, in the vendor's words or close to them	cost	src
lazy trees	Stable and on by default. Vendor claim: "3x or more" less wall time, "20x or more" less disk for many flake evaluations. Rolled out from opt-in in 3.5.2 (May 2025) to 5% in 3.7.0 and 20% in 3.8.0.	free	[1, 4]
parallel evaluation	Spreads evaluation over cores. Vendor claim: "50% or more" less evaluation time.	free	[1]
flake schemas	Lets a flake tell Nix how to show and check output types Nix does not know.	free	[1]
determinate-nixd	A daemon that schedules garbage collection (keep at least 30 GB or 5–20% free, "urgent" below 5%), logs in to FlakeHub with tokens or OIDC from GitHub Actions, GitLab, Buildkite and others, exports macOS Keychain certificates, and upgrades Nix to the version "advised by Determinate Systems".	free	[5]
installer + Actions	`install.determinate.systems`; `nix-installer-action` installs Determinate Nix by default, and installing upstream Nix "isn't a supported configuration" that "we may remove". Also `determinate-nix-action`, `flake-checker-action`, `update-flake-lock`, `flakehub-cache-action`, `DeterminateSystems/ci`.	free	[7, 10]
Magic Nix Cache	Caches Nix builds in the GitHub Actions cache. Deprecated for 1 February 2025 when GitHub changed its cache API, revived 13 June 2025 (v11 of the Action). Works only "between runs of a specific workflow in a specific repository".	free	[11, 12]
flake-checker	Checks `flake.lock`: nixpkgs on a supported branch, younger than 30 days, owned by the NixOS org. Apache-2.0.	free	[13]
FlakeHub	A registry: semver for flakes, publish from trusted CI with `flakehub-push` (inputs include `directory` for a flake in a subdirectory, `tag`, `rolling`, `visibility`), `fh resolve` / `fh apply` for pre-resolved store paths. Public flakes are free; private flakes are paid.	free (public)	[6, 14, 15]
FlakeHub Cache	A binary cache with access by Git repository and JWT, no key management. "Available only on paid plans."	paid	[8]
nixpkgs cooldown	Since 15 June 2026, `nixpkgs-weekly` takes an upstream revision only after seven days, and Determinate Nix users get it by default through the bare `nixpkgs` registry entry.	free	[17]
Secure Packages	A curated, signed nixpkgs with a CVE SLA (critical 7 days, high 15, medium 45, low 90), CycloneDX SBOMs with VEX (FlakeBOM), a policy check (FlakeAudit) and a FIPS variant. Access only for organisations granted it.	paid, contact sales	[9]
compliance + support	SOC 2 Type II, FedRAMP High (product page footer), MDM roll-out on macOS, enterprise support.	paid	[1]

Table 1. No public price list was found: the pricing pages fetched for this study returned 404 or no content, and the docs say only "free", "paid plans" or "contact sales". This table says no more than that.

03 where picasso already depends on Determinate

place in picasso	what it uses	state on 12 Oct 2026
the workstation	Determinate Nix on Ubuntu 24.04; README's setup step 1 runs `curl … install.determinate.systems/nix | sh -s -- install` (README.md, "Ubuntu users")	current
`labs/cuda` `setup-cuda-cache`	Writes the nixos-cuda substituter to `/etc/nix/nix.custom.conf`, the file Determinate Nix `!include`s; AGENTS.md's safety rules name it as the only file a script may write	matches the vendor's rule [3]
`.github/workflows/ci.yml`, `pages.yml`	`DeterminateSystems/nix-installer-action@v23` in five jobs; no Nix cache step, so every run substitutes from cache.nixos.org	current; no cache
`labs/publisher/action.yml`	`nix-installer-action@v16` and `magic-nix-cache-action@v9`, used by ww-lab and h0ffmann/h0ffmann to build PDFs	**stale**: v9 is from 21 January 2025, before the revival [11, 12]; whether it still caches was not checked
lab locks	All six labs follow `github:NixOS/nixpkgs/nixos-unstable`; lock dates 11–22 September 2026	**five of six** are older than flake-checker's 30-day threshold [13]

Two consequences follow. First, picasso already accepts Determinate as its Nix: if the installer drops upstream Nix, as its README says it may [7], nothing in picasso breaks, but the choice is no longer a choice. Second, picasso already follows the vendor's configuration rule (one custom file), so adopting more of the free tooling costs little.

04 side by side

dimension	Determinate Systems	picasso today	picasso by April 2027 (§05)
what it is	A company: Nix distribution, registry, cache, curated packages, support [1]	One person's six lab flakes + a legacy NixOS root + a site [16]	Same; no change in kind
Nix itself	Maintains a fork with lazy trees, parallel eval, schemas [2]	Uses Determinate Nix; maintains no Nix	Same. Measure lazy trees on the labs (D7)
install	Installer, Actions, MDM, NixOS / nix-darwin modules [1, 3, 7]	README points at the vendor installer	The `determinate` NixOS module in the root, once the machine moves back to NixOS (D6)
distribution of flakes	FlakeHub: semver, `fh resolve`, private flakes [6, 14]	`github:h0ffmann/picasso?dir=labs/<lab>`, sparse submodule, composite Action; releases are git tags	Also on FlakeHub as public flakes, one per lab, on each `v*` tag (D3)
versioning	Semver on FlakeHub; rolling or tagged [15]	One repo-wide semver tag (`just release`) + Zenodo DOI	Same tags drive FlakeHub; DOI stays picasso's own
binary cache	FlakeHub Cache (paid); Magic Nix Cache (free, per repo) [8, 12]	None in CI; Magic Nix Cache v9 in the publisher Action	Publisher on a current release or without it; measure before adding one to CI (D1)
lock hygiene	flake-checker, update-flake-lock, 7-day cooldown [13, 17]	CI fails a lock behind its flake; bumps are by hand	flake-checker report + a weekly bump PR per lab, taking only revisions at least 7 days old (D2, D4)
supply chain	Signed packages, CVE SLA, SBOM + VEX, FIPS (paid) [9]	Nothing beyond the lock	A best-effort CycloneDX SBOM per lab on each release, from open tools; no SLA (D5)
reproducibility	Reproducible inputs; no public rebuild-check product found	Whitepaper experiment: an image that `nix-build --check` rebuilds bit-for-bit	A CI job that rebuilds each lab's checks with `--rebuild` (D7)
domain toolchains	None; general platform	WAVEWATCH III, pandoc/TeX PDFs, lint, ai-jail agents, CUDA torch, presenting	Same labs, the reason picasso exists
operations	Managed GC, OpenTelemetry, Sentry, macOS certificates [1, 5]	Uses determinate-nixd's GC on the workstation (default)	No change
compliance	SOC 2 Type II, FedRAMP High, EU CRA material [1]	None, and none claimed	None. Out of scope
people and cost	A team; paid tiers for cache, private flakes, Secure Packages	One person; free runners; free tiers only	Same. Every item in §05 is free

Table 2. The left column cites the vendor; the middle column was read from picasso at `140dc91`; the right column is a proposal, not a commitment.

05 what picasso can do in six months



Each item follows the lab contract in picasso's AGENTS.md: a lab stays self-contained, every check builds in the sandbox, CI runs with `--no-update-lock-file`. Nothing here makes one lab depend on another, or on FlakeHub: a consumer that pins `github:h0ffmann/picasso?dir=…` keeps working.

D1 · fix the publisher Action (October–November 2026)

Bump `nix-installer-action` from v16 to the v23 CI already uses, and either move `magic-nix-cache-action` to a release after the June 2025 revival (v11 or later) or remove it. First, read the cache step's log in a recent ww-lab or h0ffmann/h0ffmann run to see whether v9 caches at all. **Effort:** one PR, plus the two consumers' next pin bump. **Cost:** none.

D2 · flake-checker in CI as a report (November 2026)

Add `flake-checker-action` to the `labs` matrix with its default exit status 0, so it writes a summary and fails nothing. Today it would flag five labs for age. It gives a visible number for a question the lab cards on the profile README already ask (the lock date). **Effort:** a few lines in `ci.yml`. **Cost:** none.

D3 · publish each lab to FlakeHub on a release tag (December 2026–January 2027)

A job in `release.yml` with a six-row matrix that runs `flakehub-push` with `directory: labs/<lab>`, `tag: v…`, `visibility: public`. Consumers can then write `https://flakehub.com/f/h0ffmann/<lab>/0.1` and get semver ranges, while the GitHub URL keeps working. **Open questions:** one repo-wide tag gives every lab the same version even when it did not change; the flake name must start with the GitHub org (`h0ffmann/…`); publishing needs `id-token: write`. The owner decides, since publishing is outward-facing and a published release stays. **Effort:** one PR and a dry run on an `unlisted` flake. **Cost:** none for public flakes.

D4 · a weekly lock bump with a cooldown (January 2027)

`update-flake-lock` on a weekly schedule, one PR per lab, so the existing CI gate decides whether a bump is safe. To copy the vendor's seven-day cooldown [17] without changing the input's owner (which flake-checker and the lab contract both expect to be NixOS), the job can pin `nixos-unstable` to the newest revision that is at least seven days old instead of the tip. Switching the labs to `DeterminateSystems/nixpkgs-weekly` is the other option; it gets the cooldown free but makes every lab depend on FlakeHub, which the owner should decide. **Effort:** small; the seven-day pick is a short script with a `--self-test`, per AGENTS.md. **Cost:** CI minutes only.

D5 · a best-effort SBOM per lab (February 2027)

Generate a CycloneDX SBOM of each lab's `lib.tools` closure on each release and attach it to the GitHub release, from open-source tools in the lab's own nixpkgs (for example `sbomnix`, if the pinned nixpkgs carries it; not checked yet). This is **not** Secure Packages: no signing beyond cache.nixos.org's, no VEX curation, no SLA. It is a list a consumer can scan. **Effort:** medium. **Cost:** none.

D6 · the determinate NixOS module in the root (when the machine moves)

AGENTS.md allows the root to be brought to the current NixOS release while the workstation moves back to NixOS. The vendor's `determinate` NixOS module writes `nix.custom.conf` from `nix.settings` [3], so `setup-cuda-cache`'s job becomes a few lines of configuration on that machine. **Depends on:** the human, the hardware and `nixos-generate-config`; no agent runs `nixos-rebuild`. **Cost:** none.

D7 · measure, don't quote (March 2027)

Two small measurements that turn vendor claims into picasso's own numbers: evaluation time and disk use of `nix flake check` per lab with lazy trees on and off, and a CI job that rebuilds each lab's `checks.*` with `nix build --rebuild` and reports which outputs are not bit-identical, as the whitepaper's experiment B did for one image. **Effort:** medium. **Cost:** CI minutes; the rebuild job runs weekly, not on every push, to stay inside free runner time.

#	item	when	effort	needs the owner for
D1	publisher Action on current Determinate Actions	Oct–Nov 2026	small	merging; consumers' pin bump
D2	flake-checker report in CI	Nov 2026	small	merging
D3	labs on FlakeHub as public flakes	Dec 2026–Jan 2027	small–medium	the decision to publish
D4	weekly lock bump with 7-day cooldown	Jan 2027	small	nixos-unstable or nixpkgs-weekly
D5	SBOM per lab on each release	Feb 2027	medium	merging
D6	determinate NixOS module in the root	when the machine moves	small	the hardware and the rebuild
D7	lazy-trees and rebuild measurements	Mar 2027	medium	reading the result

Table 3. April 2027 is left for review: which items held, which to drop. The order puts the two items that fix existing debt (D1, D2) first.

what picasso will not do

- **No SLA, no CVE deadlines, no FIPS, no SOC 2.** They need a team that answers within 24 hours; one person cannot honestly promise that.
- **No private cache or private flakes.** Both are paid on FlakeHub, and the labs are public.
- **No Nix fork and no patches to Nix.** picasso uses the Nix it is given.
- **No dependency of a lab on FlakeHub** unless the owner chooses it in D4: the lab contract says a lab references nothing outside itself, and a GitHub URL is the floor every consumer can rely on.

06 distribution: through marola, and into ouroboric-dev

Determinate's answer to distribution is a registry (FlakeHub) and a cache. picasso has two closer channels: marola-dev, which already consumes two labs, and ouroboric-dev, where the whitepaper proposed a `labs/factory`. This section reads what each channel uses today and what would make it better.

how marola gets picasso today

in marola-dev/marola	what it pins	problem
`flake.nix` input `lint`	`github:h0ffmann/nix-config?dir=labs/lint`, locked at `64219511` (16 August 2026)	old repo name (works by GitHub's redirect); follows `main`, not a release
`flake.nix` input `agentic`	`github:h0ffmann/nix-config/labs/agentic?dir=labs/agentic`	**pinned to a stacked branch**, last touched 13 September 2026, that picasso's AGENTS.md says merges into `main`, not into consumers
marola-devkit `v0.8.3`	takes `lint` and `agentic` through `follows`	every marola repo gets the labs through the devkit, so the devkit is the real distribution point
`.github/workflows/profile-activity.yml`	`h0ffmann/nix-config/.github/workflows/profile-ping.yml@main`	a reusable workflow at `@main`, unpinned

Table 4. Read from marola-dev/marola's `flake.nix`, `flake.lock` and workflows on 12 October 2026. picasso has no row in marola's REPOS.md, which lists marola-dev's own repositories.

E1 · release tags as the contract with marola (November 2026)

picasso already makes semver tags (`just release`, v0.1.0). marola's inputs move from `main` and a branch to `github:h0ffmann/picasso/v0.x.y?dir=labs/<lab>`, and the reusable workflow to `@v0.x.y`. A bump becomes a reviewed PR in marola, the rule its own AGENTS.md already states ("a consumer moves to a new producer version by bumping its pin in a PR"). **Effort:** one PR in marola-devkit, then one in marola. **Needs:** marola's agent-ready gate: someone files the issue.

E2 · the devkit as the one place marola pins picasso

Because marola-devkit already re-exports the labs, pinning picasso once there, and having marola's own `lint` and `agentic` inputs follow the devkit, removes the second pin that can drift. This is marola's decision and lives in marola-devkit, not in picasso.

E3 · FlakeHub for semver ranges (with D3)

Once D3 publishes the labs, a consumer can ask for `h0ffmann/lint/0.1` and take any compatible release, which is the ergonomic win FlakeHub sells [6]. Plain GitHub tags stay valid, so this is an option for marola, not a requirement.

E4 · ouroboric-dev: offer a lab, do not move the repo

Becoming "a repository member" of ouroboric-dev can mean three things. They are listed from least to most commitment:

option	what changes	cost and risk
a. a consumer pin	An ouroboric repo adds `inputs.factory.url = "github:h0ffmann/picasso?dir=labs/factory"` (whitepaper P1)	none for picasso; ouroboric decides; MIT code can be used in an AGPL project (whitepaper §8)
b. a lab handed over	`labs/factory` is copied into an ouroboric-dev repository and maintained there; the lab contract makes this a copy, not a split	picasso loses nothing; ouroboric gets a lab under its own licence and review
c. picasso moves	The whole repository is transferred to the ouroboric-dev org	high: the site's hard rule, the Zenodo record, the profile lab cards, marola's and ww-lab's pins and the personal workstation root all assume `h0ffmann/picasso`

Table 5. Membership is ouroboric-dev's decision, not picasso's. Nothing here was asked of ouroboric-dev yet. The A2A channel in picasso's AGENTS.md (`para:ouroboric` issues) is the place to ask.

The recommendation is **a** now and **b** if ouroboric takes the lab into its factory. Option **c** mixes a personal workstation configuration into an organisation's repository and gains nothing that **b** does not. It fits only if picasso stops being personal, which is a different project.

#	item	when	effort	decided by
E1	marola pins picasso release tags, new repo name	Nov 2026	small	marola (an issue, then agent-ready)
E2	one pin, in marola-devkit	with E1	small	marola-devkit
E3	FlakeHub semver ranges	after D3	none extra	each consumer
E4	ouroboric-dev: a pin, then maybe a lab handed over	after whitepaper P1	small–medium	ouroboric-dev, asked through A2A

07 risks and limits

- **Vendor dependence.** picasso's install path, CI and publisher Action all go through one company's tooling. The installer already calls upstream Nix unsupported [7]. The mitigation is cheap: the labs are plain flakes and still evaluate with upstream Nix, and D3 adds FlakeHub as a second address, not a replacement.
- **Vendor numbers.** "3x", "20x" and "50%" in Table 1 are the vendor's claims for some flakes. They were not measured on picasso; D7 is how to find out.
- **Prices.** No public price list was found for the paid tiers, so no paid option is costed here.
- **Bus factor of one.** Every item is sized so that it can stop half done without breaking a consumer.
- **Snapshot.** Determinate ships often (3.5 to 3.8 in two months of 2025 [4]); read this study as of 12 October 2026.

08 conclusion

Determinate Systems builds the road; picasso is a small workshop next to it. The workshop already uses the road, and two of its signs are out of date. In six months, without paying for anything, picasso can fix those signs, add the free lock checks, appear on FlakeHub under semver, copy the idea of a cooldown, and produce two things a general platform does not: domain toolchains that build in CI, and its own measured rebuild results. Those are the parts worth the owner's evenings. For distribution, the cheapest win is closer to

home: marola pins picasso's release tags in one place, and ouroboric-dev borrows a lab before anyone talks about moving a repository.

09 references

All fetched on 12 October 2026.

1. Determinate Systems. *Determinate Nix* (product page). https://determinate.systems/products/nix/
2. DeterminateSystems/nix-src, README ("released under LGPL v2.1"). https://github.com/DeterminateSystems/nix-src
3. Determinate Systems. *Determinate Nix* documentation (downstream distribution; `nix.custom.conf`; NixOS and nix-darwin modules). https://docs.determinate.systems/determinate-nix/
4. Determinate Systems. *Lazy trees*; changelogs 3.5.2, 3.7.0, 3.8.0. https://docs.determinate.systems/determinate-nix/lazy-trees/, https://determinate.systems/blog/changelog-determinate-nix-370, https://determinate.systems/blog/changelog-determinate-nix-380
5. Determinate Systems. *Determinate Nixd*. https://docs.determinate.systems/determinate-nix/determinate-nixd/
6. Determinate Systems. *FlakeHub* documentation. https://docs.determinate.systems/flakehub/
7. DeterminateSystems/nix-installer-action, README ("installs Determinate Nix by default"; upstream "isn't a supported configuration"). https://github.com/DeterminateSystems/nix-installer-action
8. Determinate Systems. *FlakeHub Cache* ("available only on paid plans"). https://docs.determinate.systems/flakehub/cache/
9. Determinate Systems. *Determinate Secure Packages*. https://docs.determinate.systems/secure-packages/
10. Determinate Systems. *Determinate in GitHub Actions*. https://docs.determinate.systems/guides/github-actions/
11. DeterminateSystems/magic-nix-cache-action, releases v9 (21 Jan), v10 (10 Jun), v11 (16 Jun). https://github.com/DeterminateSystems/magic-nix-cache-action/releases
12. Determinate Systems. *Bringing back the Magic Nix Cache Action*, 13 June 2025; and *Magic Nix Cache free tier EOL*. https://determinate.systems/blog/bringing-back-magic-nix-cache-action, https://determinate.systems/blog/magic-nix-cache-free-tier-eol
13. DeterminateSystems/flake-checker, README (checks, 30-day default, Apache-2.0). https://github.com/DeterminateSystems/flake-checker
14. DeterminateSystems/flakehub-push, README (inputs). https://github.com/DeterminateSystems/flakehub-push
15. Determinate Systems. *Publishing to FlakeHub*; *Private flakes*. https://docs.determinate.systems/flakehub/publishing/, https://docs.determinate.systems/flakehub/private-flakes
16. h0ffmann/picasso at `140dc91` and marola-dev/marola (`flake.nix`, `flake.lock`, `.github/workflows/profile-activity.yml`, AGENTS.md) on 12 October 2026: AGENTS.md, README.md, .github/workflows/ci.yml, labs/publisher/action.yml, labs/*/flake.lock, labs/cuda/README.md. https://github.com/h0ffmann/picasso
17. G. Christensen. *Nixpkgs cooldown*, Determinate Systems blog, 15 June 2026. https://determinate.systems/blog/nixpkgs-cooldown/